SENTINEL

A mutex, two semaphores and a lock order

Prelab 4

Status and deadline

This prelab is **required**, and it has **no marks**. Submit it on Brightspace by **Wednesday, December 2, 2026, by midnight**. The Lab 4 session is on Friday, December 4, 2026.

The deliverable of Lab 4 is the code that these questions describe: the Lab 4 Brief[1] states it. The course marks the deliverable out of 80, and it gives 4% of the course grade. Lab 4 has no oral exam: a video, out of 20, gives 4%. Together they give 8%.

Instructions

In Lab 4, the centre of Lab 3 gets its protection: a mutex around the track picture, two semaphores and a mutex around the queue, and one order for two locks.

- Do this prelab alone, and write the answers in your own words.
- Each question names the lecture and the manual pages that you need. If the pages are not on your machine, install them with `sudo apt install manpages manpages-dev`.
- You do not need the code of Lab 4. Part A names the lectures and the files that the questions use, and each other fact is in the question.

Part A. How to write the code of Lab 4

Lab 4 asks for three kinds of code. Learn each one here, before the session. `sync.h`[2] gives the steps of each function, and each starter file in `src/libsync/src/` repeats them above the function.

A mutex. Lecture 4.6 of the course, Mutex, shows each call that Lab 4 uses: `pthread_mutex_init()`, `pthread_mutex_lock()`, `pthread_mutex_unlock()` and `pthread_mutex_destroy()`. Your `safe_track_update` takes the lock, calls `track_update` of Lab 3, and releases the lock. One fact that the lecture does not say: a `pthread_*` function RETURNS its error number, and does not set `errno`. Your code sets `errno` to that number, as the steps say.

Two semaphores. Lecture 3.3, Semaphores, shows `sem_init()`, `sem_wait()`, `sem_post()` and `sem_destroy()`, and the producer and the consumer of `prodCons.c`: two semaphores, `semEmpty` and `semFull`, and a mutex. Your `sem_queue` is the same pattern, with two differences:

- The buffer of the lecture is a stack. Yours is your FIFO ring of Lab 3: the push calls your `queue_push`, and the pop calls your `queue_pop`, inside the mutex.

[1] The Lab 4 Brief, `docs/lab4/lab4_brief.pdf` in your repository. It states the contract of Lab 4, its ideas, its hand-in and the video.

[2] `src/libsync/include/sentinel/sync.h`, in your repository. It states what each function of Lab 4 does, in order, with each of its errors.

- `sem_wait()` can return -1 with `EINTR` when a signal comes. That is not an error: call it again, as `read()` in Lab 1.

One lock order. Lecture 4.8, Deadlocks, gives the four conditions of a deadlock and the order of the resources. Your `resources_acquire_both` takes two locks by rank, the lower rank first, whatever order the caller names them in. The C operator `?:` picks the lower of two numbers in one line: `first = (a < b) ? a : b;`.

Q1. What a mutex must cover

Read Lecture 4.6 and `man 3 pthread_mutex_lock`. Two threads call `add()` at the same time. `total` and `lock` are shared; `lock` was made with `pthread_mutex_init`.

```
int add(struct item *it)                    /* version A */
{
    pthread_mutex_lock(&lock);
    if (it == NULL) {
        return -1;
    }
    total = total + it->value;
    pthread_mutex_unlock(&lock);
    return 0;
}
```

(a) Version A has a fault on one path. Which call of `add()`, with which argument, causes it? What happens to the other thread after that call?

(b) Version B moves `pthread_mutex_lock(&lock)` down, to the line after the addition, and keeps the rest. What can `total` lose now, and why?

(c) Write the correct version: where is the check of `it`, where is the lock, and where is the unlock?

Q2. Two semaphores and a queue of 3 slots

Read the producer and the consumer of `prodCons.c` in Lecture 3.3, and `man 3 sem_wait`. A queue of 3 slots starts with `empty` = 3 and `full` = 0. A push does `sem_wait(&empty)`, puts the report in, and `sem_post(&full)`. A pop does `sem_wait(&full)`, takes the report out, and `sem_post(&empty)`. Each thread does these steps in order.

(a) Do these seven operations: push A, push B, pop, push C, push D, push E, pop. The push of E is in a thread of its own, and the last pop is in another thread. Give `empty` and `full` after each operation. Which operation waits, and what wakes it? Which reports wait in the queue at the end?

(b) A student writes `sem_post(&empty)` at the end of the push, in place of `sem_post(&full)`. What does a pop on that queue do?

Q3. Two locks, two orders

Read Lecture 4.8. Two threads share the mutexes `sensor` and `link`:

```
thread 1:  lock(sensor);  lock(link);   ...   unlock(link);  unlock(sensor);
thread 2:  lock(link);    lock(sensor); ...   unlock(sensor); unlock(link);
```

(a) Give an order of the steps of the two threads after which neither can continue.

(b) Name the four conditions of a deadlock. Thread 2 changes to take `sensor` first, then `link`. Which condition does that remove? Why do the other three stay?

Q4. Which lock fixes which fault

Two runs of Lab 3, with 8 workers on 8 processors and no lock, gave these lines (the Lab 3 Brief, Sections 3.5 and 6.1):

```
processed         20722 of 20705 queued
queue says        18446744073709112535 reports still wait, in a queue of 8192
```

Section 3.5 of the Lab 3 Brief also says that two workers can lose an update of `t->total_reports`, the total of the track picture.

(a) For each of the three faults, name the shared data that was damaged, and the part of Lab 4 that protects it: the mutex of `safe_track_update`, or the mutex and the semaphores of `sem_queue`.

(b) With the protection of Lab 4, what must the final program print in its ledger, and in the line of R4? Section 6.1 of the Lab 4 Brief shows both.